

Notebook Manufacturing Scheduling Optimization Model

Context & Background

This model schedules production orders for a **notebook manufacturing facility**. Each order (job) is a specific notebook product — for example, "A101-Plastic Spiral-72 pages-A4-Lined". The factory produces notebooks by routing them through a fixed sequence of machines depending on the binding type. The goal is to find the best assignment of jobs to production routes and sequence them on shared machines to meet delivery deadlines as efficiently as possible.

This is a **Mixed-Integer Programming (MIP)** model, intended to be solved with a solver like CPLEX or CBC.

Sets

Symbol	Name	Description
I	Jobs	All production orders to be scheduled (198 jobs in the full dataset)
J	Machines	All machines in the factory (13 machines)
R	Routes	All defined production routes (7 routes)

Machines (J)

ID	Machine Name
Bielom1	Bielomatik 1 (paper folding/cutting machine)
Bielom2	Bielomatik 2 (paper folding/cutting machine)
PolarBicak	Polar Blade (paper trimmer)

ID	Machine Name
Kugler	Kugler (spiral coil inserter)
Plasticol	Plasticol (plastic spiral closer)
Kore	Kore (alternative plastic spiral closer)
Autobind	Autobind 500 (double-wire binding machine)
CWH	CWH Double (alternative double-wire binder)
ManSpiral	Manuel Spiral (manual spiral binding station)
Juki	Juki (thread sewing machine)
Shrink1	Shrink1+Koli (shrink-wrap + boxing, line 1)
Shrink2	Shrink2+Koli (shrink-wrap + boxing, line 2)
MSK	MSK Etiket (labeling machine)

Production Routes (R)

Every job must travel through exactly one route. A route defines the **ordered sequence of machines** the job must pass through.

Route	Binding Type	Machine Sequence (in order)
r1	Wire Stitch	Bielom2 → Shrink1 → MSK
r2	Plastic Spiral (via Plasticol)	Bielom1 → PolarBicak → Kugler → Plasticol → Shrink2 → MSK
r3	Plastic Spiral (via Kore)	Bielom1 → PolarBicak → Kore → Shrink2 → MSK
r4	Double Metal Spiral (via Autobind)	Bielom1 → PolarBicak → Kugler → Autobind → Shrink2 → MSK
r5	Double Metal Spiral (via CWH)	Bielom1 → PolarBicak → CWH → Shrink2 → MSK

Route	Binding Type	Machine Sequence (in order)
r6	Manual Spiral	Bielom1 → PolarBicak → Kugler → ManSpiral → Shrink2 → MSK
r7	Thread Stitch	Bielom1 → PolarBicak → Juki → Shrink2 → MSK

Important: Routes r2/r3 are both valid for plastic spiral jobs — the model chooses which one. Similarly r4/r5 for double metal spiral. This is the core route-selection decision.

Parameters

Processing Times

Each job's processing time on a machine depends on:

1. Which route it's assigned to
2. Whether that machine is the **bottleneck machine** on that route

The bottleneck machine on each route determines the pace of production (it takes the full cycle time). All other machines on the route take 25% of that time (they are support operations, much faster).

Route	Bottleneck Machine	Cycle time (sec/unit)
r1	Bielom2	2 sec
r2	Plasticol	15 sec
r3	Kore	16.5 sec
r4	Autobind	9 or 17 sec (depends on product size)
r5	CWH	15 or 23 sec (depends on product size)

Route	Bottleneck Machine	Cycle time (sec/unit)
r6	ManSpiral	17 sec
r7	Juki	15 sec

Processing time for job i on machine j via route r (in minutes):

$p(i, r, j) = \text{cycle_time}(i, r) / 60$ if j is the bottleneck machine on route r

$p(i, r, j) = \text{cycle_time}(i, r) / 60 \times 0.25$ for all other machines on route r

The total time a job occupies a machine = $p(i, r, j) \times \text{qty}(i)$ where $\text{qty}(i)$ is the order quantity in units.

Setup Times

When two jobs process sequentially on the same Bielomatik machine (Bielom1 or Bielom2), a **40-minute setup** is required if they have different paper ruling types (lined ↔ squared, lined ↔ plain, or squared ↔ plain). No setup is required if consecutive jobs have the same paper type.

$s(i, i', j) = 40$ minutes if jobs i and i' differ in paper type and $j \in \{\text{Bielom1}, \text{Bielom2}\}$

$s(i, i', j) = 0$ otherwise

Due Dates

Each job has a delivery deadline $d(i)$ expressed in **minutes from today**. It is computed as:

$d(i) = (\text{delivery_date} - \text{today}) \times 1200$ minutes/day

The factory operates a 1200-minute workday (20 hours).

Machine Capacities

$\text{cap}(\text{Bielom1}) = 4800$ min/day (two shifts)

$\text{cap}(\text{Bielom2}) = 4800$ min/day (two shifts)

$\text{cap}(\text{Shrink1}) = 2400$ min/day

$\text{cap}(\text{Shrink2}) = 2400$ min/day

$\text{cap}(\text{MSK}) = 2400$ min/day

cap(all others) = 1200 min/day (single shift)

Route-Machine Compatibility

$ar(r, j) = 1$ if route r uses machine j , else 0. (Defined by the route table above.)

Job-Route Compatibility

$compat(i, r) = 1$ if job i 's product type can be processed on route r , else 0.

- Wire Stitch jobs → only $r1$
 - Plastic Spiral jobs → $r2$ or $r3$
 - Double Metal Spiral jobs → $r4$ or $r5$
 - Manual Spiral jobs → only $r6$
 - Thread Stitch jobs → only $r7$
-

Decision Variables

Assignment Variables

Variable	Type	Meaning
$x(i, r)$	Binary {0,1}	= 1 if job i is assigned to route r
$y(i, j)$	Continuous [0,1]	= 1 if job i is processed on machine j (derived from x via K2)

Sequencing Variables

Variable	Type	Meaning
$z(i, i', j)$	Binary {0,1}	= 1 if job i is processed before job i' on machine j

Note: z is only defined for pairs (i, i') that can actually share a machine (i.e., both jobs have a compatible route that uses machine j). All other z values are fixed to 0 to reduce model size.

Time Variables

Variable	Type	Meaning
$C(i, j)$	Continuous ≥ 0	Completion time of job i on machine j (in minutes)
$C_i(i)$	Continuous ≥ 0	Final completion time of job i (= max over all machines it uses)
$T_i(i)$	Continuous ≥ 0	Tardiness of job i = $\max(0, C_i(i) - d(i))$
$\text{delta}(i)$	Binary $\{0,1\}$	= 1 if job i is late (misses its deadline)
$C_{\max}$	Continuous ≥ 0	Makespan = the latest completion time across all jobs

Objective Function

Minimize a weighted combination of four penalty terms:

$$\text{Minimize } Z = \alpha \cdot C_{\max} + \beta \cdot \sum_i T_i(i) + \gamma \cdot \sum_i \text{delta}(i)$$

Subject to: $\alpha + \beta + \gamma = 1$ (weights must sum to 1)

Note: The original model document mentions a 4th term $\lambda \cdot \sum \text{setup costs}$, but the GMS implementation uses only 3 terms. The implemented weights are: $\alpha = 0.10$, $\beta = 0.60$, $\gamma = 0.30$.

What each term penalizes:

Term	Weight	Penalizes	Why it matters
$\alpha \cdot C_{\max}$	0.10	The time until the last job finishes	Shorter makespan = factory finishes the full batch sooner

Term	Weight	Penalizes	Why it matters
$\beta \cdot \sum T_i$	0.60	Weighted total tardiness (minutes late $\times$ priority weight)	High-priority orders that are late are penalized more heavily
$\gamma \cdot \sum \delta_i$	0.30	Number of late jobs (regardless of how late)	Even 1-minute late counts — minimizes how many customers are affected

Constraints

K1 — Each Job Must Be Assigned to Exactly One Route

$$\sum_{r \in R} x(i, r) = 1 \quad \text{for all } i \in I \quad (\text{only over compatible routes})$$

Plain English: Every job must go down exactly one production line. It cannot split across two routes simultaneously, and it cannot be left unassigned.

K2 — Route-Machine Consistency (Linking Constraint)

$$y(i, j) = \sum_{r: ar(r,j)=1, \text{ compat}(i,r)=1} x(i, r) \quad \text{for all } i \in I, j \in J$$

Plain English: If job i is assigned to route r , and route r uses machine j , then job i must be processed on machine j . This constraint links the route decision (x) to the machine usage variable (y). You don't choose machine assignments separately — they are fully determined by the route choice.

K3 — Machine Capacity Constraint

 **Note:** K3 appears in the original document but is **not explicitly implemented** in the GMS model. The GMS model handles timing implicitly through the completion time constraints (K4/K5/K8). The capacity is effectively enforced by the sequencing constraints — machines can only do one job at a time.

If implemented explicitly:

$$\sum_i p(i,j) \cdot y(i,j) \leq \text{cap}(j) \quad \text{for all } j \in J$$

Plain English: The total processing time of all jobs assigned to machine j cannot exceed its daily shift capacity.

K4 — Minimum Completion Time (Lower Bound on C)

$$C(i, j) \geq \sum_{\{r: \text{compat}(i,r)=1, \text{ar}(r,j)=1\}} p(i, r, j) \cdot \text{qty}(i) \cdot x(i, r) \quad \text{for all } i, j$$

Plain English: Job i 's completion time on machine j must be at least as large as its own processing time on that machine. This is a lower bound — $C(i,j)$ could be larger if j has to wait for prior jobs to finish.

K5 — No-Overlap (Disjunctive) Constraint

This is the core sequencing constraint. It ensures no two jobs overlap on the same machine.

For every pair of jobs (i, i') that both use machine j :

$$C(i, j) \geq C(i', j) + p(i, r, j) \cdot \text{qty}(i) + s(i', i, j) - M \cdot (1 - z(i, i', j)) \quad [i \text{ goes after } i']$$

$$C(i', j) \geq C(i, j) + p(i', r, j) \cdot \text{qty}(i') + s(i, i', j) - M \cdot z(i, i', j) \quad [i' \text{ goes after } i]$$

Plain English: Either job i finishes before job i' starts (plus any setup time needed), OR job i' finishes before job i starts. The binary variable $z(i, i', j)$ selects which ordering is active. The Big-M term deactivates the constraint for the non-chosen ordering.

Why both constraints? Exactly one of $z=1$ or $z=0$ will be chosen. When $z=1$, the first constraint is active (i goes after i'). When $z=0$, the second constraint is active (i' goes after i). This guarantees a definite ordering without overlap.

K6 — Makespan and Job Completion Time Definitions

$$C_i(i) \geq C(i, j) \quad \text{for all } j \text{ that job } i \text{ uses} \quad [\text{job's completion} = \text{max over all its machines}]$$

$$C_{\max} \geq C_i(i) \quad \text{for all } i \quad [\text{makespan} = \text{max over all jobs}]$$

Plain English: A job is "done" only when it has cleared its last machine. The makespan is the time when the very last job in the entire schedule completes.

K7 — Tardiness Definitions

$T_i(i) \geq C_i(i) - d(i)$ for all i [tardiness = how many minutes past deadline]

$T_i(i) \geq 0$ [tardiness cannot be negative]

$\delta(i) \geq (C_i(i) - d(i)) / M$ [delta = 1 if any tardiness exists]

Plain English: If a job finishes after its due date, T_i records by how many minutes. δ is a binary flag that becomes 1 whenever a job is late (used for counting the number of late jobs in the objective).

K8 — In-Route Machine Sequence (Precedence)

$C(i, jj) \geq C(i, j) + p(i, r, j) \cdot \text{qty}(i) \cdot x(i, r)$

for all i, r, j, jj where:

- $\text{compat}(i, r) = 1$ (job i can use route r)
- $\text{ar}(r, j) = 1$ (route r uses machine j)
- $\text{ar}(r, jj) = 1$ (route r uses machine jj)
- $\text{seq}(r, jj) = \text{seq}(r, j) + 1$ (jj is the next machine after j in route r)

Plain English: Within a route, machines must be visited in order. A job cannot start on the next machine in its route until it has finished on the current one. For example, a plastic spiral job must complete on PolarBicak before it can start on Kugler.

K9 — Variable Domains

$x(i, r), \delta(i), z(i, i', j) \in \{0, 1\}$ (binary)

$y(i, j), C(i, j), C_i(i), T_i(i), C_{\max} \geq 0$ (non-negative continuous)

$p(i, j), d(i), w(i), \text{cap}(j) \geq 0$ (non-negative parameters)

$x(i, r) = 0$ if $\text{compat}(i, r) = 0$ (incompatible route-job pairs locked to 0)

Implementation Notes (from GMS file)

These are decisions already made in the GAMS implementation that an agent implementing this model should be aware of:

Solver settings (CPLEX):

- 4 threads
- MIP emphasis: finding feasible solutions fast
- Optimality gap tolerance: 10% ($\text{epgap} = 0.10$)
- Time limit: 3600 seconds (1 hour)
- Final solution allowed to be within 5% of optimal ($\text{optcr} = 0.05$)

Scale:

- 198 jobs, 13 machines, 7 routes
- Time unit: **minutes**
- 1 workday = 1200 minutes (also used to convert due dates from calendar days)
- Upper bound on all C variables: 2,500,000 minutes

λ term (setup cost in objective): The original document includes a 4th objective term for total setup costs. The GMS implementation drops this term ($\lambda = 0$) and uses only $\alpha=0.10$, $\beta=0.60$, $\gamma=0.30$.

z variable sparsity: The $z(i, i', j)$ variable is only instantiated for job pairs that can actually compete for the same machine. This dramatically reduces the number of binary variables.

Summary: What the Model Decides

Given a set of notebook production orders with quantities, due dates, and product types, the model decides:

1. **Which route** each job takes (e.g., plastic spiral via Plasticol or via Kore)

2. **In what order** jobs are sequenced on each shared machine
3. **When** each job starts and finishes on each machine

To minimize a weighted combination of: how long total production takes, how much total lateness accumulates, and how many orders miss their deadlines.